Cloud Project Proposal - Anime Recommendation Platform

Student Name: Soluchi Fidel-Ibeabuchi
Email: soluchi.fidelibeab@bison.howard.edu
Course: CS363 - Large Scale Programming
Date: January 26, 2026
Existing Repository: github.com/soluchi/animerec

Project Title: Cloud-Native Anime Recommendation System on Google Cloud Platform

---

Executive Summary

I propose to transform my existing anime recommendation system project into a production-grade, cloud-native application deployed on Google Cloud Platform (GCP). This project will demonstrate comprehensive cloud architecture principles by migrating a local Python-based recommendation engine into a scalable, multi-tier web application with serverless compute, managed databases, CI/CD pipelines, and enterprise-grade monitoring.

The project encompasses the full cloud development lifecycle: architecture design, infrastructure provisioning, application deployment, security implementation, cost optimization, and operational monitoring—all core competencies equivalent to the AWS Cloud Practitioner certification.

Current Project State

Existing Implementation (github.com/soluchi/animerec)

My current anime recommendation system is a Python-based application that provides personalized anime suggestions using collaborative filtering and content-based algorithms. The system currently:

- Processes anime datasets (titles, genres, ratings, user preferences)
- Implements recommendation algorithms (similarity-based matching)
- Runs locally with Python scripts
- Stores data in local CSV/JSON files

Limitations of Current Implementation

- **No scalability**: Cannot handle multiple concurrent users
- **No persistence**: User preferences not saved between sessions
- **No accessibility**: Requires local Python environment to run
- **No API**: Cannot be integrated with other applications
- **No monitoring**: No visibility into performance or usage
- **No deployment**: Cannot be shared or accessed publicly

Project Objectives

Primary Learning Objectives

1. **Cloud Architecture Design**: Design and implement a multi-tier, cloud-native architecture
2. **Serverless Computing**: Deploy application logic using Google Cloud Functions
3. **Managed Databases**: Migrate data to Cloud Firestore and Cloud Storage
4. **API Development**: Create RESTful APIs for recommendation services
5. **Frontend Development**: Build responsive web interface for end users
6. **Security Implementation**: Implement authentication, authorization, and data encryption
7. **CI/CD Pipeline**: Automate testing and deployment with GitHub Actions
8. **Cost Management**: Monitor and optimize cloud spending
9. **Observability**: Implement logging, monitoring, and alerting
10. **Infrastructure as Code**: Define infrastructure using Terraform

Technical Competencies Demonstrated

- Cloud computing fundamentals (IaaS, PaaS, SaaS)
- Serverless architecture patterns
- Database design and management
- API design and RESTful principles
- Web application security
- DevOps and automation
- Performance optimization
- Cost analysis and budgeting

Proposed Architecture

System Architecture Overview

```
┌──────────────────────────────────────────────────────────────┐
│                          USERS                               │
└──────────────────────────┬───────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│              FRONTEND TIER (Presentation)                    │
│  - Firebase Hosting (Static Web App)                         │
│  - Cloud CDN (Global Distribution)                           │
│  - React.js Single Page Application                          │
└──────────────────────────┬───────────────────────────────────┘
                           │
                           ▼
┌──────────────────────────────────────────────────────────────┐
│              APPLICATION TIER (Business Logic)               │
│  - Cloud Functions (Serverless API)                          │
│    • GET /api/recommendations/{user_id}                      │
```

```
│    • POST /api/ratings                                          │
│    • GET /api/anime/search                                      │
│    • GET /api/anime/trending                                    │
│  - Cloud Run (Containerized ML Model - Optional)                │
│  - Firebase Authentication (User Management)                    │
└─────────────────────────────────────────────────────────────────┘
                          │
                          │
                          ▼

┌─────────────────────────────────────────────────────────────────┐
│                  DATA TIER (Persistence)                        │
│  - Cloud Storage (Anime Dataset - CSV/JSON)                     │
│  - Cloud Firestore (User Profiles, Ratings, Preferences)        │
│  - Cloud Memorystore (Redis Cache - Optional)                   │
└─────────────────────────────────────────────────────────────────┘
                          │
                          ▼

┌─────────────────────────────────────────────────────────────────┐
│             OPERATIONS TIER (Monitoring & DevOps)               │
│  - Cloud Monitoring (Metrics & Dashboards)                      │
│  - Cloud Logging (Centralized Logs)                             │
│  - Cloud Build + GitHub Actions (CI/CD)                         │
│  - Cloud Billing (Cost Management & Alerts)                     │
└─────────────────────────────────────────────────────────────────┘
```

Component Details

1. Frontend Tier

- **Firebase Hosting**: Hosts static React.js web application
- **Cloud CDN**: Distributes content globally for low latency
- **Features**:
  - User registration and login interface
  - Anime search and browse functionality
  - Personalized recommendation display
  - Rating and favorites management
  - Responsive design (mobile and desktop)

2. Application Tier

- **Cloud Functions (Node.js/Python)**: Serverless API endpoints
  - `getRecommendations`: Returns personalized anime suggestions
  - `submitRating`: Processes user ratings
  - `searchAnime`: Searches anime by title, genre, rating
  - `getTrending`: Returns popular anime based on recent ratings
- **Firebase Authentication**: Manages user identity and sessions
- **Cloud Run (Optional)**: Hosts ML model for advanced recommendations

3. Data Tier

- **Cloud Storage**:
  - Stores anime master dataset (CSV files)
  - Stores static assets (images, posters)
- **Cloud Firestore**: NoSQL database for:
  - User profiles and preferences
  - User ratings and watch history
  - Real-time data synchronization
- **Cloud Memorystore (Optional)**: Redis cache for frequently accessed data

4. Operations Tier

- **Cloud Monitoring**: Tracks performance metrics, uptime, errors
- **Cloud Logging**: Centralized log aggregation and analysis
- **Cloud Build**: Automated build and deployment
- **GitHub Actions**: CI/CD pipeline orchestration
- **Cloud Billing**: Cost tracking and budget alerts

Technical Implementation Plan

Phase 1: Foundation & Setup (Week 1)

Objectives: Set up GCP environment and migrate core functionality

Tasks:

- [ ] Create GCP project and enable required APIs
- [ ] Set up billing with budget alerts ($20-30/month limit)
- [ ] Configure Firebase project
- [ ] Create service accounts and IAM roles
- [ ] Set up GitHub repository structure
- [ ] Initialize Terraform configuration

Deliverables:

- GCP project configured with proper permissions
- Firebase project initialized
- Terraform infrastructure skeleton
- Updated repository structure

Phase 2: Data Migration (Week 2)

Objectives: Migrate anime dataset and establish data layer

Tasks:

- [ ] Upload anime dataset to Cloud Storage buckets
- [ ] Design Firestore data schema:

```
users/
  {userId}/
    - profile: {name, email, joinDate}
    - preferences: {favoriteGenres[], excludedGenres[]}
    - ratings: {animeId: rating}
    - watchlist: [animeIds]

anime/
  {animeId}/
    - title, genres, rating, description, imageUrl
    - aggregatedRating, ratingCount
```

- ☐ Create data loading scripts (Python)
- ☐ Implement data validation
- ☐ Set up Firestore security rules
- ☐ Configure Cloud Storage bucket policies

Deliverables:

- Populated Cloud Storage with anime data
- Firestore database with schema documentation
- Data migration scripts
- Security rules implemented

Phase 3: Backend API Development (Week 3)

Objectives: Build serverless API using Cloud Functions

API Endpoints:

1. Recommendation Endpoint

```python
# Cloud Function: get_recommendations
import functions_framework
from google.cloud import firestore
from recommendation_engine import generate_recommendations

@functions_framework.http
def get_recommendations(request):
    """
    GET /api/recommendations/{user_id}?limit=10
    Returns personalized anime recommendations
    """
    user_id = request.args.get('user_id')
    limit = int(request.args.get('limit', 10))

    db = firestore.Client()
    user_data = db.collection('users').document(user_id).get()
```

```
    recommendations = generate_recommendations(user_data, limit)

    return {
        'user_id': user_id,
        'recommendations': recommendations,
        'generated_at': datetime.now().isoformat()
    }
```

2. Rating Endpoint

```
@functions_framework.http
def submit_rating(request):
    """
    POST /api/ratings
    Body: {user_id, anime_id, rating, timestamp}
    Stores user rating and updates recommendation model
    """
    data = request.get_json()
    db = firestore.Client()

    # Store rating
    db.collection('users').document(data['user_id']) \
      .collection('ratings').document(data['anime_id']).set(data)

    # Update anime aggregate rating
    update_anime_rating(data['anime_id'])

    return {'status': 'success', 'rating_id': data['anime_id']}
```

3. Search Endpoint

```
@functions_framework.http
def search_anime(request):
    """
    GET /api/anime/search?query=naruto&genre=action&minRating=8
    Returns filtered anime results
    """
    query = request.args.get('query', '')
    genre = request.args.get('genre')
    min_rating = float(request.args.get('minRating', 0))

    results = search_anime_database(query, genre, min_rating)
    return {'results': results, 'count': len(results)}
```

4. Trending Endpoint

```python
@functions_framework.http
def get_trending(request):
    """
    GET /api/anime/trending?period=week&limit=20
    Returns trending anime based on recent ratings
    """
    period = request.args.get('period', 'week')
    limit = int(request.args.get('limit', 20))

    trending = calculate_trending_anime(period, limit)
    return {'trending': trending, 'period': period}
```

Tasks:

- ☐ Implement recommendation algorithm (collaborative filtering)
- ☐ Create Cloud Functions for each endpoint
- ☐ Set up CORS configuration
- ☐ Implement error handling and logging
- ☐ Write unit tests for functions
- ☐ Configure function authentication

Deliverables:

- 4+ deployed Cloud Functions
- API documentation (OpenAPI/Swagger)
- Unit test suite
- Postman collection for testing

Phase 4: Frontend Development (Week 4)

Objectives: Build user-facing web application

Technology Stack:

- React.js 18+
- Material-UI or Tailwind CSS
- Firebase SDK (Auth + Firestore)
- Axios for API calls

Pages & Features:

1. **Landing Page**
    - Hero section with app description
    - Featured/trending anime carousel
    - Call-to-action (Sign Up/Login)
2. **Authentication**
    - Sign up with email/password

    - Login with Firebase Auth
    - Google OAuth integration

3. **Browse Page**
    - Search bar with filters (genre, rating, year)
    - Grid view of anime cards
    - Pagination
    - Sort options (popularity, rating, recent)

4. **Anime Detail Page**
    - Anime information (title, genres, synopsis, rating)
    - Rate anime (1-10 stars)
    - Add to watchlist
    - Similar anime recommendations

5. **Recommendations Page**
    - Personalized recommendations based on user history
    - Explanation of why recommended
    - Filter by genre

6. **Profile Page**
    - User statistics (anime watched, average rating)
    - Favorite genres
    - Rating history
    - Watchlist management

Tasks:

- [ ] Initialize React project with Create React App or Vite
- [ ] Implement Firebase Authentication
- [ ] Create reusable UI components
- [ ] Integrate API calls to Cloud Functions
- [ ] Implement responsive design
- [ ] Add loading states and error handling
- [ ] Deploy to Firebase Hosting

Deliverables:

- Fully functional React web application
- Deployed to Firebase Hosting
- Mobile-responsive design
- Public URL for access

Phase 5: Security Implementation (Week 5)

Objectives: Implement comprehensive security measures

Security Layers:

1. **Authentication & Authorization**
   - Firebase Authentication for user identity
   - JWT token validation in Cloud Functions
   - Role-based access control (user, admin)
2. **Data Security**
   - Firestore security rules:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid ==
userId;
    }
    match /anime/{animeId} {
      allow read: if true;
      allow write: if request.auth != null &&

get(/databases/$(database)/documents/users/$(request.auth.uid)).data.role ==
'admin';
    }
  }
}
```

   - Cloud Storage bucket permissions
   - Encryption at rest (default) and in transit (HTTPS)
3. **API Security**
   - Rate limiting on Cloud Functions
   - Input validation and sanitization
   - CORS configuration
   - API key management using Secret Manager
4. **Network Security**
   - HTTPS only (Firebase Hosting enforces)
   - Cloud Armor for DDoS protection (optional)

Tasks:

- [ ] Implement Firebase Auth in frontend
- [ ] Write Firestore security rules
- [ ] Configure Cloud Function authentication
- [ ] Set up Secret Manager for API keys
- [ ] Implement rate limiting

- ☐ Conduct security audit

Deliverables:

- Security implementation documentation
- Firestore security rules
- Authentication flow diagram
- Security testing report

Phase 6: CI/CD Pipeline (Week 5-6)

Objectives: Automate testing and deployment

GitHub Actions Workflow:

```yaml
name: Deploy to GCP

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with:
          python-version: '3.11'
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Run tests
        run: pytest tests/
      - name: Lint code
        run: flake8 functions/

  deploy-functions:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v3
      - name: Authenticate to GCP
        uses: google-github-actions/auth@v1
        with:
          credentials_json: ${{ secrets.GCP_SA_KEY }}
      - name: Deploy Cloud Functions
        run: |
```

```
            gcloud functions deploy get-recommendations \
              --runtime python311 \
              --trigger-http \
              --allow-unauthenticated \
              --region us-central1

    deploy-frontend:
      needs: test
      runs-on: ubuntu-latest
      if: github.ref == 'refs/heads/main'
      steps:
        - uses: actions/checkout@v3
        - name: Install Node.js
          uses: actions/setup-node@v3
          with:
            node-version: '18'
        - name: Build React app
          run: |
            cd frontend
            npm install
            npm run build
        - name: Deploy to Firebase Hosting
          uses: FirebaseExtended/action-hosting-deploy@v0
          with:
            repoToken: ${{ secrets.GITHUB_TOKEN }}
            firebaseServiceAccount: ${{ secrets.FIREBASE_SA_KEY }}
            channelId: live
```

Tasks:

- ☐ Set up GitHub Actions workflow
- ☐ Configure GCP service account
- ☐ Add GitHub secrets
- ☐ Implement automated testing
- ☐ Set up staging environment
- ☐ Configure deployment approvals

Deliverables:

- Automated CI/CD pipeline
- Deployment documentation
- Test coverage report

Phase 7: Monitoring & Optimization (Week 6)

Objectives: Implement observability and optimize performance

Monitoring Setup:

1. **Cloud Monitoring Dashboards**

    - API response times
    - Error rates
    - Request volume
    - Function execution times
    - Database read/write operations

2. **Logging**
    - Structured logging in Cloud Functions
    - Error tracking and alerting
    - User activity logs

3. **Alerting**
    - Error rate exceeds 5%
    - API latency > 2 seconds
    - Daily cost exceeds $2
    - Function failures

Performance Optimization:

- ☐ Implement caching for popular recommendations
- ☐ Optimize database queries (indexes)
- ☐ Compress frontend assets
- ☐ Implement lazy loading for images
- ☐ Reduce Cloud Function cold start times

Cost Optimization:

- ☐ Analyze Cloud Billing reports
- ☐ Identify cost drivers
- ☐ Implement resource quotas
- ☐ Optimize function memory allocation
- ☐ Set up budget alerts

Tasks:

- ☐ Create monitoring dashboards
- ☐ Configure log-based metrics
- ☐ Set up alerting policies
- ☐ Run performance tests
- ☐ Conduct cost analysis

Deliverables:

- Monitoring dashboard screenshots
- Performance benchmark report
- Cost analysis with optimization recommendations
- Alert configuration documentation

Deliverables Summary

1. **Live Application**

- **Public URL**: https://animerec-[project-id].web.app
- **Features**:
  - User authentication
  - Anime search and browse
  - Personalized recommendations
  - Rating system
  - Responsive design

2. **GitHub Repository**

- **URL**: github.com/soluchi/animerec
- **Contents**:
  - Backend code (Cloud Functions)
  - Frontend code (React app)
  - Infrastructure as Code (Terraform)
  - CI/CD configuration (GitHub Actions)
  - Comprehensive README
  - API documentation

3. **Architecture Documentation**

- System architecture diagram
- Data flow diagrams
- Database schema documentation
- API endpoint documentation (Swagger/OpenAPI spec)
- Security architecture document

4. **Technical Reports**

a. Cost Analysis Report

- Monthly cost breakdown by service
- Cost optimization strategies implemented
- Projected costs at scale
- Budget recommendations

b. Security Implementation Report

- Authentication mechanisms
- Authorization policies
- Data encryption methods
- Security testing results
- Compliance considerations

c. Performance Benchmarking Report

- API response time analysis
- Database query performance
- Frontend load times
- Scalability testing results
- Optimization recommendations

d. DevOps Documentation

- CI/CD pipeline architecture
- Deployment procedures
- Rollback strategies
- Monitoring and alerting setup

5. **Presentation Materials**

- Demo video (5-10 minutes)
- Slide deck explaining:
  - Problem statement
  - Architecture design
  - Key technical decisions
  - Challenges and solutions
  - Lessons learned
- Live demonstration

Technology Stack

Google Cloud Platform Services

Category	Service	Purpose
Compute	Cloud Functions	Serverless API endpoints
	Cloud Run (Optional)	Containerized ML model hosting
Storage	Cloud Storage	Anime dataset storage
	Cloud Firestore	User data and ratings database
	Cloud Memorystore (Optional)	Redis caching layer
Networking	Cloud CDN	Global content delivery
	Cloud Load Balancing	Traffic distribution
Security	Firebase Authentication	User identity management
	Secret Manager	API key and credential storage
	Cloud Armor (Optional)	DDoS protection
Operations	Cloud Monitoring	Metrics and dashboards
	Cloud Logging	Centralized logging

Category	Service	Purpose
	Cloud Billing	Cost tracking
DevOps	Cloud Build	Automated builds
	Artifact Registry	Container image storage
Frontend	Firebase Hosting	Static website hosting

Development Tools

- **Backend**: Python 3.11 (Cloud Functions), Node.js 18+
- **Frontend**: React.js 18, Material-UI/Tailwind CSS
- **Database**: Firestore SDK, Cloud Storage SDK
- **Testing**: pytest, Jest, Cypress
- **CI/CD**: GitHub Actions, Cloud Build
- **IaC**: Terraform
- **Monitoring**: Cloud Monitoring, Cloud Logging
- **Documentation**: Swagger/OpenAPI, Markdown

Comparison to AWS Cloud Practitioner

This project demonstrates equivalent depth and scope to AWS Cloud Practitioner certification by covering all core cloud concepts:

Cloud Concepts

AWS Concept	GCP Implementation	Project Component
Compute	Lambda → Cloud Functions	API endpoints
Storage	S3 → Cloud Storage	Dataset storage
Database	DynamoDB → Firestore	User data
CDN	CloudFront → Cloud CDN	Frontend distribution
Authentication	Cognito → Firebase Auth	User management
Monitoring	CloudWatch → Cloud Monitoring	Observability
Logging	CloudWatch Logs → Cloud Logging	Log aggregation
Deployment	CloudFormation → Terraform	Infrastructure as Code
CI/CD	CodePipeline → Cloud Build	Automated deployment
Cost Management	Cost Explorer → Cloud Billing	Budget tracking
Security	IAM → Cloud IAM	Access control
API Gateway	API Gateway → Cloud Endpoints	API management

AWS Concept	GCP Implementation	Project Component
Caching	ElastiCache → Memorystore	Performance optimization

Key Competencies Demonstrated

☑ **Cloud Fundamentals**

- Understanding of cloud service models (IaaS, PaaS, SaaS)
- Multi-tier architecture design
- Scalability and elasticity concepts

☑ **Security & Compliance**

- Identity and access management
- Data encryption
- Network security
- Security best practices

☑ **Billing & Pricing**

- Cost estimation
- Resource optimization
- Budget management
- Cost monitoring

☑ **Architecture Design**

- High availability
- Fault tolerance
- Disaster recovery
- Performance optimization

☑ **Core Services**

- Compute (serverless)
- Storage (object, database)
- Networking (CDN, load balancing)
- Monitoring and logging

Timeline

6-Week Implementation Schedule

Week	Phase	Key Milestones
1	Foundation	GCP setup, Firebase init, repo structure
2	Data Layer	Dataset migration, Firestore schema, security rules
3	Backend	Cloud Functions deployed, API tested

Week	Phase	Key Milestones
4	Frontend	React app developed, Firebase integrated
5	Security & DevOps	Auth implemented, CI/CD pipeline active
6	Optimization & Documentation	Monitoring setup, reports completed

Detailed Weekly Goals

Week 1: Foundation & Setup

- ☑ GCP project created
- ☑ Billing configured with alerts
- ☑ Firebase project initialized
- ☑ Repository restructured
- ☑ Terraform basics implemented

Week 2: Data Migration

- ☑ Anime dataset uploaded to Cloud Storage
- ☑ Firestore schema designed and implemented
- ☑ Security rules written
- ☑ Data loading scripts completed

Week 3: Backend API Development

- ☑ 4 Cloud Functions deployed
- ☑ Recommendation algorithm implemented
- ☑ API documentation written
- ☑ Unit tests passing

Week 4: Frontend Development

- ☑ React app structure created
- ☑ All pages implemented
- ☑ Firebase Auth integrated
- ☑ Deployed to Firebase Hosting

Week 5: Security & CI/CD

- ☑ Security rules enforced
- ☑ GitHub Actions pipeline operational
- ☑ Automated testing implemented
- ☑ Secret management configured

Week 6: Monitoring & Polish

- ☑ Dashboards created
- ☑ Performance optimizations applied
- ☑ Cost analysis completed
- ☑ Documentation finalized

- ☑ Demo video recorded

Risk Management

Potential Risks & Mitigation Strategies

Risk	Impact	Probability	Mitigation
GCP free tier limits exceeded	High	Medium	Set strict budget alerts ($20), optimize resource usage
API cold start latency	Medium	High	Implement Cloud Run for critical functions, use min instances
Data migration complexity	Medium	Low	Test with small dataset first, validate data integrity
Security vulnerabilities	High	Low	Regular security audits, follow GCP best practices
CI/CD pipeline failures	Low	Medium	Comprehensive testing, staging environment
Scope creep	Medium	Medium	Stick to defined deliverables, defer "nice-to-have" features
Learning curve with GCP	Low	High	Leverage GCP documentation, tutorials, and quickstarts

---

Budget Estimation

Monthly Cost Breakdown (Estimated)

Service	Usage	Monthly Cost
Cloud Functions	100K invocations, 256MB	$0 (free tier)
Cloud Storage	5GB storage, 10GB egress	$0.10
Firestore	50K reads, 20K writes, 1GB storage	$0 (free tier)
Firebase Hosting	10GB transfer	$0 (free tier)
Cloud CDN	10GB egress	$0.80
Cloud Monitoring	Basic metrics	$0 (free tier)
Cloud Build	120 build minutes/day	$0 (free tier)
Firebase Authentication	<10K MAU	$0 (free tier)
Total		**~$1-2/month**

Cost Optimization Strategies:

- Leverage GCP free tier extensively

- Set budget alerts at $20/month
- Use Cloud Functions instead of always-on instances
- Implement caching to reduce database reads
- Optimize function memory allocation
- Monitor and right-size resources

Success Criteria

Project will be considered successful if:

☑ **Functionality**

- Application is fully deployed and accessible via public URL
- All API endpoints function correctly
- User authentication works reliably
- Recommendations are generated accurately
- Database operations are performant

☑ **Architecture**

- Multi-tier architecture properly implemented
- Serverless design patterns followed
- Security best practices applied
- High availability achieved (>99% uptime)

☑ **DevOps**

- CI/CD pipeline operational
- Automated testing implemented
- Infrastructure as Code working
- Monitoring and alerting configured

☑ **Documentation**

- Complete technical documentation
- API documentation (Swagger)
- Architecture diagrams
- Setup and deployment guides
- Cost and performance reports

☑ **Demonstration**

- Live demo of working application
- Presentation explaining architecture
- Clear articulation of design decisions
- Evidence of cloud best practices

Conclusion

This project transforms my existing anime recommendation system into a production-grade, cloud-native application that comprehensively demonstrates cloud computing principles equivalent to the AWS Cloud Practitioner certification. By implementing a multi-tier architecture on Google Cloud Platform, I will gain hands-on experience with:

- **Serverless computing** (Cloud Functions)
- **Managed databases** (Firestore, Cloud Storage)
- **Frontend hosting** (Firebase Hosting with CDN)
- **Authentication and security** (Firebase Auth, IAM)
- **DevOps practices** (CI/CD, Infrastructure as Code)
- **Monitoring and optimization** (Cloud Monitoring, cost management)

The project scope is substantial yet achievable within the 6-week timeline, with clear milestones and deliverables. The existing codebase provides a strong foundation, while the cloud migration introduces significant technical complexity and learning opportunities.

Most importantly, this project demonstrates **real-world cloud engineering skills**: designing scalable architectures, implementing security best practices, automating deployments, monitoring production systems, and optimizing costs—all essential competencies for modern software engineering roles.

Appendix

A. Useful GCP Resources

- GCP Free Tier: https://cloud.google.com/free
- Cloud Functions Quickstart: https://cloud.google.com/functions/docs/quickstart
- Firestore Documentation: https://firebase.google.com/docs/firestore
- Firebase Hosting Guide: https://firebase.google.com/docs/hosting
- Terraform GCP Provider: https://registry.terraform.io/providers/hashicorp/google/latest/docs

B. Learning Resources

- Google Cloud Skills Boost (Free): https://www.cloudskillsboost.google/
- GCP Architecture Framework: https://cloud.google.com/architecture/framework
- Firebase Samples: https://github.com/firebase/quickstart-js

C. Contact Information

- **Student**: [Your Name]
- **Email**: [Your Email]
- **GitHub**: github.com/soluchi
- **Existing Project**: github.com/soluchi/animerec

I am confident this project will demonstrate comprehensive cloud computing competencies equivalent to the AWS Cloud Practitioner certification while delivering a functional, production-ready application. I look forward to your feedback and approval to proceed.

Respectfully submitted,
Soluchi Fidel-Ibeabuchi
January 26, 2026